Container-Specific Service Mesh-Based System for Mitigating Lateral Movement Attacks

Geonwoo Yoon, Jinmyeong Shin, Jae-Seok Kim, Seunghyuk Kim, Jaeyoung Jeong, and Yoon-Ho Choi, *Member, IEEE*

***Abstract*—The dynamic nature of containers within a Kubernetes cluster substantially expands the potential attack surface. In particular, lateral movement attacks enable adversaries to compromise additional subsystems after gaining initial access. To defend against lateral movement attacks, most anomaly detection methods rely on offline learning using system call data accumulated over a period of time. However, such offline learning methods struggle to capture the unique system call patterns of individual containers and lack adaptability to changes caused by frequent container updates. To address these limitations, we propose a new service mesh-based system for online learning of container-specific system call patterns observed under cloud-native microservice environments. The proposed service mesh-based system consists of three key functional processes as follows: (i) a zero-copy-based system call collection process, which leverages eBPF for efficient monitoring; (ii) an anomaly detection and container isolation process, which employs lightweight machine learning models, and leverages a proxy container for swift traffic control; and (iii) a container-specific online learning process, which continuously updates anomaly detection models by learning observed system call sequences. From the experimental results, we show that the proposed zero-copy-based system call collection process significantly improves system call collection speeds by as much as 4.5 times and shows lower CPU usage by as much as half compared to other state-of-the-art methods. Furthermore, the container-specific online learning process consistently outperforms offline learning approaches across various system call datasets and maintains stable detection performance by continuously adapting to behavioral changes caused by container updates over time.**

***Index Terms*—Lateral movement, anomaly detection, service mesh, online learning, cloud-native.**

Received 21 April 2025; revised 7 November 2025; accepted 20 January 2026. Date of publication 23 January 2026; date of current version 10 March 2026. This work was supported in part by the IITP (Institute of Information & Communications Technology Planning & Evaluation)-ITRC (Information Technology Research Center), the Korea government (MSIT, Ministry of Science and ICT) under Grant IITP-2025-RS-2023-00259967, in part by the Basic Science Research Program through the National Research Foundation of Korea (NRF), the Ministry of Education under Grant 2022R1I1A3055233, and in part by the NRF grant funded by the Korea government (MSIT) under Grant RS-2023-00217689. Recommended for acceptance by J. Hua.*(Corresponding author: Yoon-Ho Choi.)*

Geonwoo Yoon, Jae-Seok Kim, Seunghyuk Kim, Jaeyoung Jeong, and Yoon-Ho Choi are with the School of Computer Science and Engineering, Pusan National University, Busan 46241, Republic of Korea (e-mail: glove2275@pusan.ac.kr; tjr3117@pusan.ac.kr; senghyuk66@pusan.ac.kr; jyjung0211@pusan.ac.kr; yhchoi@pusan.ac.kr).

Jinmyeong Shin is with the Department of Computer Science and Engineering, University of California, Merced, CA 95343 USA (e-mail: jinmyeongshin@ucmerced.edu).

Digital Object Identifier 10.1109/TCC.2026.3657486

I. Introduction

As virtualization technologies, such as containers and virtual machines (VMs), have evolved, the cloud-native approach [1] has emerged as a new paradigm for developing and operating modern applications. The cloud-native approach enables organizations to build and manage scalable applications consisting of numerous microservices. Microservices are an architectural method of developing software applications as a distributed set of independently deployable and manageable modules. The distributed modules, which run various application services, are typically deployed as containers in cloud-native environments. Each microservice serves a distinct function and interacts with other microservices via well-defined application programming interfaces (APIs). This modular approach simplifies the management of large-scale applications. With these capabilities, cloud-based applications are now widely adopted across various service domains, such as manufacturing, healthcare, finance, logistics, and so on.

Despite these advantages, the growing service coverage and increasing sophistication of cloud-based applications have made the cloud computing environment more complex to manage. This complexity arises from the need to configure security policies tailored to each container's workload, access permissions, and network connectivity requirements [2], [3], [4]. It not only increases the difficulty of system management but also broadens the potential attack surface. As a result, the presence of multiple interconnected containers creates vulnerabilities that are frequently exploited by lateral movement attacks [5], [6]. Lateral movement attacks are a common tactic used by adversaries to traverse cloud environments by exploiting network-level access and insufficient isolation between containers. In such attacks, an adversary initially compromises a container or microservice and then expands access by probing adjacent services, escalating privileges, and targeting critical resources.

As a representative approach to mitigating lateral movement attacks, service mesh [7], [8] controls communication between containerized applications. The service mesh is a dedicated infrastructure layer that provides static security policies to control network traffic between containers. This infrastructure layer provides various features, including monitoring, logging, and traffic control capabilities. However, existing service mesh tools [9], [10] rely on static configurations, requiring users to have extensive expertise in managing complex configurations while adapting to dynamic cloud-native environments.

2168-7161 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

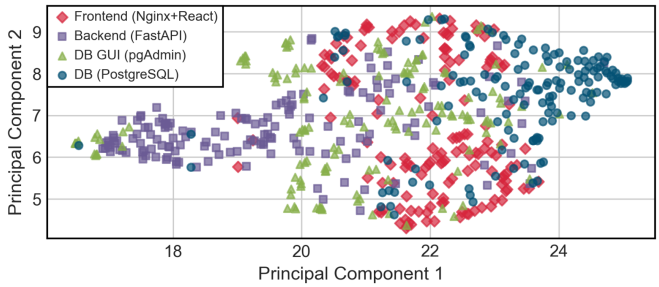


Fig. 1. Two-dimensional visualization of behavior features extracted from system call patterns across four containers in a web scenario, where high-dimensional behavior vectors were projected using Principal Component Analysis (PCA). In the legend, the preceding text indicates the role of each container, while the text in parentheses specifies the representative tools used.

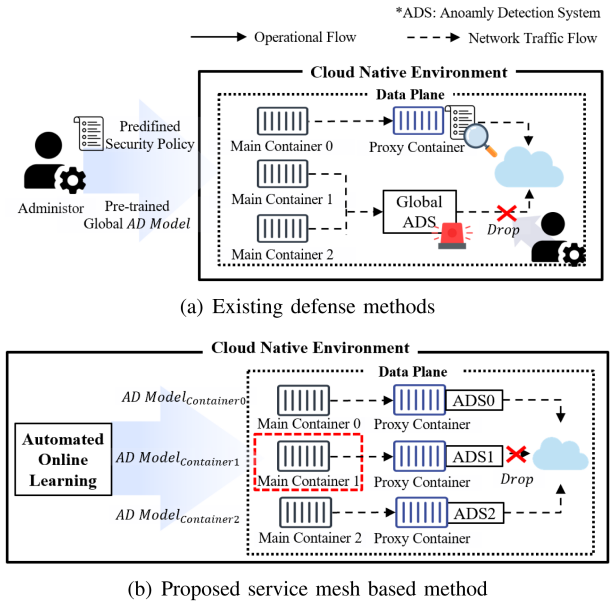


Fig. 2. Architecture and Operation differences between (a) existing systems, and (b) proposed system against lateral movement attacks.

According to a Red Hat report [11] on Kubernetes [12], a platform for managing containerized workloads and services, approximately 40% of reported security incidents were caused by misconfiguration in containers or Kubernetes. To address this issue, several studies [13], [14], [15], [16] have explored automated security configuration for cloud-native environments. However, other studies [17], [18], [19], [20] indicate that such automation remains practically infeasible in real-world deployments.

To overcome the limitations of static security policies and improve adaptability, machine learning-based approaches [21], [22], [23], [24], [25] have been applied for anomaly detection in cloud-native environments. These approaches aim to identify malicious intrusions in the early stage of attacks by analyzing system calls generated from containers. However, these approaches rely on offline learning models that are trained on accumulated datasets over a period of time. Moreover, because these models primarily focus on datasets collected from the entire cloud environment, they struggle to detect exceptional anomalies in specific types of containers. Fig. 1 illustrates a two-dimensional visualization of the feature distributions of system call patterns collected from four different containers used in this study. These containers were deployed to run a web service scenario, and each container exhibits a distinct distribution of features derived from system call patterns. This indicates that common patterns alone are insufficient for accurately detecting anomalies in cloud-native environments.

To address this challenge and robustly detect container-specific anomalies in dynamic cloud-native environments, detection approaches should not only analyze fine-grained, container-specific patterns but also effectively adapt to frequent container updates. However, offline learning approaches do not account for such environmental dynamics, and their detection accuracy tends to degrade as containers evolve and become more complex. In contrast, online learning approaches incrementally train their models with newly observed patterns, continuously adapting to individual container behavior. Thus, online learning approaches can maintain high detection accuracy, even when containers exhibit unique or evolving patterns that offline models typically struggle to capture.

Based on the advantages of the online learning approach, we propose a new service mesh-based system to monitor and analyze the runtime behavior of each container. Fig. 2 illustrates the high-level differences between the existing and proposed systems for mitigating lateral movement attacks. Existing service mesh systems rely on static security inspections based on predefined policies. Moreover, offline learning-based anomaly detection systems (ADSs) train anomaly detection (AD) models using historical data collected from the cloud environment prior to deployment, as illustrated in Fig. 2(a). In contrast, the proposed system incorporates multiple AD models, customized for individual containers, as shown in Fig. 2(b). Upon detecting anomalous system call patterns, this system isolates the compromised container to defend against lateral movement attacks. To implement this approach, we target three design goals: efficient, low-overhead system call monitoring, accurate real-time anomaly detection, and continuously adaptive models for container-specific behaviors.

Accordingly, the system is structured around three main functional processes that align with these design goals:

1) **Zero-copy-based system call collection process** monitors system calls of main containers, which execute the actual service workload, from within proxy containers introduced in our system while maintaining kernel-level container isolation. This process minimizes resource overhead and achieves a system call collection speed at least 4.5 times faster compared to existing methods.
2) **Anomaly detection and container isolation process** analyzes system calls in proxy containers. Upon detecting anomalies, the process isolates the compromised main containers. It also collects training data for online learning. Experimental results confirmed that system call

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF EXISTING APPROACHES AND PROPOSED SYSTEM. ✓: SUPPORTED; △: PARTIAL/INDIRECT; ×: NOT SUPPORTED; –: NOT APPLICABLE. FOR ISOLATION-PRESERVING MONITORING, ✓: BIDIRECTIONAL ISOLATION; △: UNIDIRECTIONAL VISIBILITY.

Research Work	Cloud-Native Environment	Container-Aware	Dynamic Adaptation	Automated Online Learning	Isolation-Preserving Monitoring	Lateral Movement Defense
Service Mesh [9], [10], [26]	✓	✓	×	×	✓	△
Monitoring [27]–[29]	✓	–	–	–	△	–
Graph Method [30]–[32]	×	×	×	×	–	×
ML-based IDS [23]–[25]	✓	×	×	×	△	×
ML-based IDS [33]	✓	×	×	×	△	✓
DeepLog [22]	×	×	✓	×	△	×
Proposed System	✓	✓	✓	✓	✓	✓

collection and anomaly detection do not introduce additional network latency compared to existing service mesh tools.

3) **Container-specific online learning process** continuously updates anomaly detection models tailored to each container to adapt to frequent container changes. Experiments demonstrated that this approach achieves superior detection performance compared to offline learning approaches.

We implement the proposed service mesh-based system in Kubernetes, a representative cloud-native platform, and present this work with a focus on its design and evaluation within Kubernetes environments. This implementation requires no kernel modifications or additional software, ensuring broad portability. In addition, to evaluate the effectiveness of the proposed online learning approach, we design Kubernetes-specific lateral movement attack scenarios and real-world service workloads.

The rest of this paper is organized as follows. Section II describes the background knowledge, related research, and motivation. In Section III, the basic assumptions and threat models are described. Section IV describes the proposed service mesh-based system. A comprehensive security and performance evaluation on a Kubernetes cluster is presented in Sections V and VI. Finally, this work is concluded in Sections VII and VIII.

II. RELATED WORK AND MOTIVATION

This section presents the background knowledge and studies relevant to this work. In particular, it discusses the limitations of existing service mesh architectures and intrusion detection systems (IDS) in dynamic cloud-native environments.

A. Preliminaries

1) Kubernetes: Kubernetes [12] is a leading open-source platform for managing containerized workloads and services. The platform offers portability and extensibility and facilitates both declarative configuration and automation. A Kubernetes cluster consists of a master node, which is responsible for managing the cluster's state, and multiple worker nodes, where containerized applications are deployed and run. The fundamental deployment unit is the Pod, the smallest deployable computing unit within Kubernetes. It groups one or more containers that share resources and are scheduled together on the same node. Among these, multi-container Pods are often used to support sidecar containers, which run alongside the main container and provide supplementary services such as logging, monitoring, security, or data synchronization without modifying the main container's core code. To support scalability and availability, Kubernetes provides built-in resources like ReplicaSets, Deployments, and StatefulSets, which create and manage Pod replicas for load balancing.

2) eBPF: Extended Berkeley Packet Filter (eBPF) [34] is a Linux kernel technology that enables sandboxed programs to run in privileged contexts, such as the kernel itself. It allows for the safe and efficient extension of kernel capabilities without the need to modify kernel source code. eBPF is primarily applied in networking, security, and observability.

In observability and security, eBPF programs are attached to kernel events such as tracepoints, kprobes, or perf events to collect runtime information with minimal disruption. Among these, system call tracepoints [35] provide an efficient, stable interface for monitoring process activities at the kernel boundary. They are statically defined hooks that trigger at system call entry and exit, invoking attached eBPF functions for real-time event observation. Tracepoints offer a stable application binary interface (ABI), enabling consistent attachment across kernel versions and low-overhead monitoring across all containers.

Building on these capabilities, we leverage system call tracepoints to design an efficient system call collection mechanism. By attaching eBPF programs to these tracepoints, system call metadata is captured directly in the kernel and transferred to user space via a shared buffer, achieving real-time, zero-copy-based system call collection with minimal overhead.

B. Container Monitoring Approaches

Container monitoring spans multiple approaches [27], [28], [29]. A common choice is node-level collection, where a privileged host agent aggregates system calls from all containers on a node. An alternative is distributed monitoring via the sidecar pattern, where each main container is paired with a monitoring container. In this work, we analyze the main container's system calls within the sidecar container. Therefore, centralized analysis after node-level collection is out of scope.

1) Node-Level Collection: In node-based collection, a privileged node agent captures system calls invoked by all containers on the node, then classifies events by main container and forwards them to the relevant sidecar container or shares them via files. However, this centralized approach incurs additional

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

copying, queuing latency, and resource overhead during the filtering and forwarding process.

2) Container-Level Collection: Container-level collection can be implemented inside the application container and forwarded to a sidecar, but this requires modifying the application image and introduces additional overhead. A common alternative is for the sidecar to collect directly, leaving the application container unmodified. In Kubernetes, enabling `shareProcessNamespace: true` merges PID namespaces across containers in a Pod, and `hostPID: true` places the container in the host PID namespace. However, this approach risks weakening isolation and expanding the attack surface. Enabling `shareProcessNamespace` allows an attacker who compromises the application container to tamper with sidecar processes and leverage their privileges. Enabling `hostPID` gives a compromised sidecar visibility into host processes, raising the likelihood of privilege escalation.

Prior efforts to mitigate sidecar risks [27], [28], [29] combine privilege reduction (e.g., read-only mounts, capability minimization) with limited boundary relaxations (e.g., specific namespace sharing or read-only filesystem access). These techniques reduce risk against insider threats but do not fully prevent the weakening of container boundaries inherent to sidecar container-based monitoring in real deployments.

C. Defense Against Lateral Movement Attack

1) Service Mesh Architecture: As the scale and complexity of containerized applications increase, along with the number of containers, ensuring visibility into network communication becomes challenging. To address this challenge, service mesh [7], [8] has emerged as a cloud-native solution. A service mesh is a dedicated software layer that manages all inter-service traffic. It ensures that reliability, observability, and security features are consistently applied across all services.

A service mesh architecture typically comprises two key components: the control plane and the data plane. The control plane functions as a central management and configuration hub. It disseminates information to the data plane, including parameters like service endpoints, routing rules, load balancing policies, and security settings. The data plane is responsible for processing data. It consists of a collection of proxies deployed as sidecars alongside each service instance. These proxies intercept and manage all network traffic between microservices. The service mesh architectural pattern is exemplified by several popular implementations, including Istio [9], Linkerd [10], and AWS App Mesh [26].

2) Lateral Movement Detection: Recent research has explored graph-based methods for detecting lateral movement attack [30], [31], [32], [36], [37], [38]. Ho et al. [30] introduced Hopper, a system that detects lateral movement by constructing a graph from enterprise logs. Hopper applies an inference algorithm and anomaly scoring to identify suspicious login paths within the graph and map suspicious activities to specific users. Khoury et al. [31] proposed Jbeil, a temporal graph neural network (GNN) system for detecting lateral movement attacks in enterprise networks. It processes authentication events by aggregating temporal node memories and computing embeddings to identify malicious movement patterns. Bowman et al. [38] presented an approach that models computer networks as authentication graphs, where nodes represent entities and edges represent authentication events. The method applies unsupervised graph learning to generate node embeddings and uses link prediction to identify anomalous authentication patterns that could indicate lateral movement. Tian et al. [32] propose CloudSEC to analyze authentication events to prevent lateral movement attacks in edge computing environments. The system constructs a directed graph based on vulnerability correlations and environmental information. Using this graph structure, it identifies malicious movement paths through connected authentication event sequences.

However, these approaches do not address the challenges of a dynamic microservice-based Kubernetes environment. Specifically, they lack the capability to handle containers and services that are rapidly created and destroyed in microservice architectures. To address these challenges, the proposed service mesh system creates a dedicated proxy container for each container and implements a dynamic traffic control mechanism. This approach enables real-time monitoring and control of container behavior while maintaining the flexibility required for microservices environments.

3) Anomaly Detection Using System Call: Numerous studies [22], [23], [24], [25], [33], [39] have explored anomaly detection through system call analysis. Du et al. [22] proposed DeepLog, which uses LSTM to model system logs as natural language sequences and to detect deviations from learned normal patterns. Chen et al. [23] introduced a CNN-based approach that encodes system call sequences into fixed-size two-dimensional representations, enabling the model to learn spatial patterns for real-time detection of malicious behavior. Wang et al. [24] presented a BiLSTM-based Variational Autoencoder (VAE) for unsupervised anomaly detection, leveraging VAE's generative capabilities and BiLSTM's ability to capture long-term dependencies. Guo et al. [39] employed an ensemble learning approach using features from system call traces via a bag-of-system-calls (BoSC) approach, combined with resource metrics. This approach constructs time-aligned feature vectors and aggregates standardized anomaly scores to detect zero-day attacks. Kim et al. [25] proposed a system that combines deduplicated embedding and zero-shot learning for efficient system call analysis, achieving improved accuracy and speed.

These approaches face limitations in cloud-native environments where containers exhibit unique system call patterns and frequent updates. The diversity across containers makes it difficult to define container-specific normal behavior, while constant application updates necessitate continuous model retraining. Our system addresses these challenges through online learning, adapting dynamically to container-specific characteristics and updates while maintaining operational efficiency.

D. Motivation

Existing approaches to system call collection and defense against lateral movement attacks suffer from security and performance limitations. Node-level collectors introduce extra copying and queuing latency during filtering and forwarding.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

Sidecar-based collection, when it relies on namespace relaxation, weakens isolation, enlarges the attack surface, and can create privilege escalation paths. Meanwhile, heterogeneous and rapidly evolving container-specific system call patterns hinder unified model robustness and degrade cluster-wide attack detection performance. Furthermore, existing offline learning methods also deteriorate as applications update.

To address these issues, we aim to: (i) enable low-overhead, container-scoped system call collection without relaxing namespaces; (ii) provide accurate, real-time anomaly detection tailored to each service; and (iii) employ automated online learning to continuously adapt models to changing container behavior. This combination targets both the efficiency and security limitations of prior collection methods while maintaining effectiveness under rapid microservice changes.

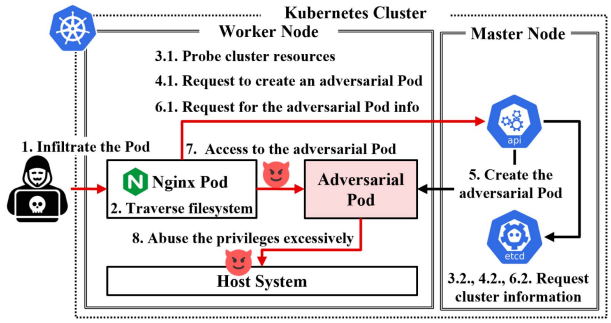


Fig. 3. A lateral movement attack scenario in a Kubernetes-based cloud-native environment, which consists of a master node and a worker node with a misconfigured Nginx container.

III. Threat Model

In this paper, we mainly focus on a network-based lateral movement attack scenario initiated by misconfiguration or vulnerability of the target containerized microservice in a cloud-native computing environment. One of the misconfigured (or vulnerable) microservices, which is exposed to external networks, can allow access from a malicious user. As a result, the malicious user can try a lateral movement attack using APIs for internal communication between cloud components (e.g., cloud operators, nodes, and containers) by exploiting misconfigured settings (or vulnerabilities).

In Kubernetes, namespaces provide a mechanism for isolating groups of resources within a single cluster. Each namespace contains service accounts, which provide unique identities for processes running in Pods. A service account's identity is crucial because containers within Pods need to authenticate with the API server to perform various operations, such as creating resources, managing configurations, and accessing cluster information.

Every Kubernetes namespace inherently includes at least one service account, typically designated as the default. If a specific service account is not determined during Pod creation, Kubernetes automatically assigns the default service account. By default, Kubernetes automatically mounts the service account's API credentials (token and certificate) into the containers for API server authentication. This default configuration often leads to unnecessary exposure of API credentials within containers. Furthermore, these service accounts are frequently configured with overly permissive roles without careful consideration of the principle of least privilege. These security misconfigurations can be exploited by attackers to escalate privileges and move laterally within the cluster if a container is compromised.

Fig. 3 illustrates a network-based lateral movement attack scenario in a Kubernetes environment exploiting a misconfigured service account and Pod on a Linux-based node. In this scenario, Nginx, a widely used web server and reverse proxy, is deployed as a Pod with the service account. The Pod inherits potentially excessive permissions from the service account, and the service account's API token is automatically mounted within the Pod. An adversary can gain initial access by exploiting vulnerabilities in Nginx, such as CVE-2024-43804, CVE-2024-23827, and other recent critical vulnerabilities [40], [41], [42]. In the case of CVE-2024-49368, Nginx UI versions prior to 2.0.0-beta.36 allowed arbitrary command execution through the logrotate functionality. Logrotate is a system utility designed to automate log file management tasks such as rotation, compression, and removal. The vulnerability occurred because Nginx UI directly passed user input to *exec.Command* without any verification checks when configuring logrotate, potentially enabling unauthorized server access.

After gaining initial access to the Nginx Pod, the adversary can retrieve the mounted service account credentials. Because the service account may have overly broad permissions within the cluster, the adversary can exploit these credentials to access the Kubernetes API. These activities may encompass querying sensitive cluster data, manipulating Pod configurations, and altering network policies.

By examining the retrieved service account's credentials and interaction with the Kubernetes API, the adversary can systematically identify the specific permissions associated with the compromised Pod's service account. If the service account possesses the authority to create Pods or modify existing privileges, the adversary can leverage these capabilities to escalate their access privileges within the cluster. The privilege escalation typically involves deploying a malicious Pod containing a backdoor image. The malicious Pod is intentionally configured with elevated privileges, such as access to host system mounts and enhanced Linux capabilities. Upon successful deployment of the malicious Pod, the adversary queries the API server to confirm the malicious Pod's deployment status and obtain the necessary information to access the malicious Pod.

Finally, the adversary establishes a connection to the malicious Pod through pre-configured channels (e.g., SSH, Telnet) embedded within the backdoor image. By exploiting host system mount points and elevated Linux capabilities, particularly through the `chroot` command, which changes a process's view to the root directory, the adversary can break out of the container's restricted environment and gain root-level access to the underlying host system. With root-level access, the adversary gains full control over the Kubernetes cluster. Such elevated privilege facilitates the execution of a wide range of malicious activities. These activities may include accessing all running

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

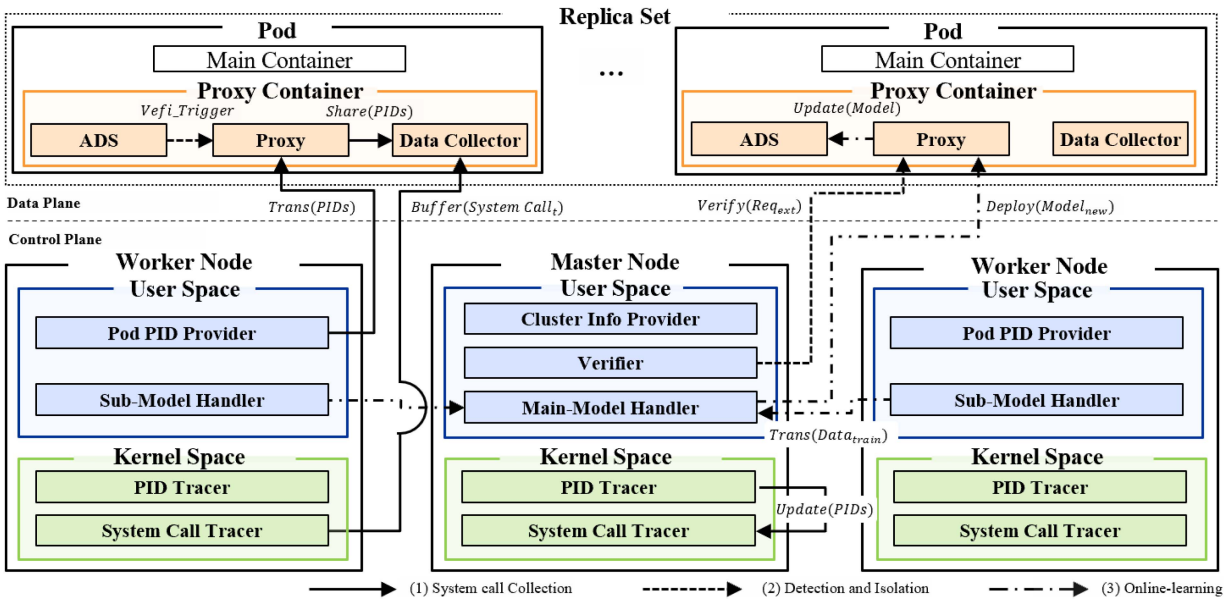


Fig. 4. Service mesh-based system architecture and the operational overview of three processes: (i) zero-copy-based system call collection process; (ii) anomaly detection and container isolation process; and (iii) container-specific online learning process.

containers, manipulating critical cluster configurations, intercepting network traffic, and potentially propagating the attack across multiple nodes within the cluster. Furthermore, with root privileges, the adversary can establish persistence in the compromised environment. Establishing persistence involves installing rootkits, creating backdoors, or manipulating system configurations, which significantly impedes subsequent detection and remediation efforts.

IV. Proposed Service Mesh-Based System

To achieve container-specific anomaly detection and compromised container isolation through online learning under lateral movement attacks, the proposed service mesh-based system consists of three main processes: (i) a zero-copy-based system call collection process, (ii) an anomaly detection and container isolation process, and (iii) a container-specific online learning process. The architecture and operational details of each process are described in the following subsections.

A. Architectural Overview

Fig. 4 illustrates the architecture and operational overview of the proposed system, which comprises three main processes. It integrates seamlessly with Kubernetes and is logically partitioned into two distinct planes: a control plane and a data plane. The control plane, comprising user space and kernel space modules, provides operational elements for the data plane. These include system calls of the main containers, which perform the actual service, and the anomaly detection models. The data plane, implemented as a set of proxy containers introduced as part of our system, performs anomaly detection through continuous monitoring of the main containers and executes isolation procedures for compromised containers through traffic control mechanisms.

The control plane's functionality is implemented on both master and worker nodes. In the master node, both the Cluster Info Provider and Verifier modules support anomaly detection and container isolation. The Cluster Info Provider module supplies essential cluster information to proxy containers, while the Verifier module performs verification on detected anomalies. The Main-Model Handler module receives data from the Sub-Model Handler modules and updates the container-specific model to orchestrate the online learning process. In each worker node, the Pod PID Provider module is responsible for initially identifying the process IDs (PIDs) of main containers and transmitting them to the associated proxy container. The Sub-Model Handler module manages and transmits the data necessary for updating anomaly detection models. The kernel spaces of each node commonly include the PID Tracer and System Call Tracer modules. Common across all nodes, the PID Tracer and System Call Tracer modules operate in the kernel space. The PID Tracer module continuously updates the list of PIDs to be tracked by monitoring process lifecycle events, while the System Call Tracer module collects system calls associated with these PIDs.

In the data plane, the proxy container serves as a core component. Each proxy container is deployed as a sidecar to a Pod alongside its main container. The sidecar architecture enables the proxy container to intercept and monitor network traffic to and from the main container. This architecture allows for fine-grained control and comprehensive visibility into main container activities. Each proxy container consists of three main modules: the Proxy module, the Anomaly Detection System (ADS), and the Data Collector module. The Proxy module handles network traffic and provides the necessary information to other modules within the proxy container. Furthermore, it verifies anomaly

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

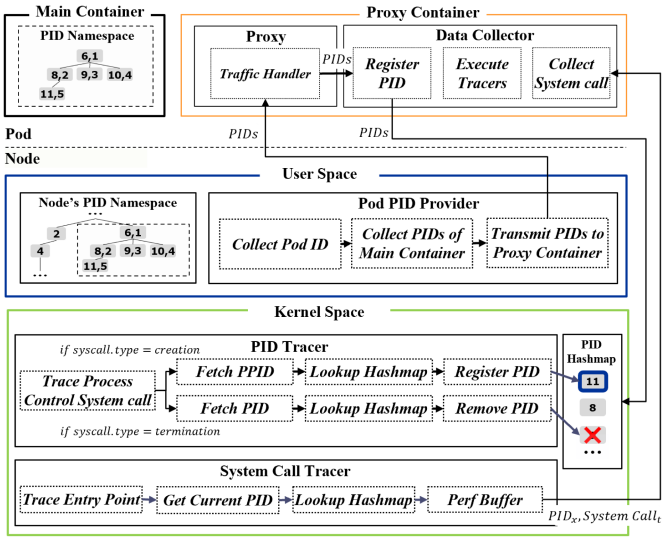


Fig. 5. Operational flow of zero-copy-based system call collection process.

detection results from the ADS and applies the proposed defense mechanisms through traffic control actions such as forwarding, relaying, or dropping traffic. The Data Collector module interacts with kernel space modules to collect system call data, which the ADS analyzes to detect abnormal behavior exhibited by the main container.

B. System Operations

This section describes the system's three main processes and the roles of the modules within each process. These processes are tightly integrated to prevent lateral movement attacks, enabling the system to continuously update container-specific models and isolate containers that exhibit abnormal behavior based on detection results.

1) Zero-Copy-Based System Call Collection: In order to monitor the main container while maintaining isolation, the proxy container requires the main container's process IDs (PIDs) in the root namespace and the capability to perform kernel monitoring through eBPF. This kernel-level capability allows the proxy container to observe processes across container boundaries, enabling comprehensive visibility into the main container's processes.

Fig. 5 illustrates the detailed design of our zero-copy-based system call collection process. The first step in monitoring the main container from the isolated proxy container is obtaining its PIDs. By utilizing the parent-child process hierarchy of container runtimes, the Pod PID Provider module identifies relevant processes in the target Pod and extracts the main container's PIDs. The Pod PID Provider module transmits these PIDs within the same Pod's proxy container, which passes them to the Data Collector module. Upon receiving the PIDs, the Data Collector module registers them into the eBPF map [43], a generic kernel data structure for sharing information between kernel and user space. This registration enables continuous kernel-level monitoring of the target processes.

Following the initial PID registration by the Data Collector module, the target PIDs must be continuously updated whenever processes in the main container are created or terminated. Therefore, the PID Tracer module is responsible for maintaining the PIDs in the eBPF map. It updates the eBPF map by actively monitoring system calls associated with process lifecycle events (e.g., fork, clone, exit, exit_group). The PID Tracer module continuously updates the eBPF map to reflect the current state in the main container. By dynamically adding new PIDs and removing PIDs of terminated processes, the PID Tracer module ensures continuous tracking of all processes in the main container.

The System Call Tracer module collects system calls by hooking into the system call tracepoint [35]. This tracepoint is a kernel instrumentation mechanism triggered whenever a system call is executed in the Linux kernel. When a system call is invoked, and this tracepoint is triggered, the module identifies the PID that initiated the system call. To determine whether the PID belongs to a main container process, the System Call Tracer module queries an eBPF map containing the set of monitored PIDs. When the PID matches one in the monitored set, the module inserts both the PID and system call number into a ring buffer [44] shared with the Data Collector module. The Data Collector continuously monitors the ring buffer to retrieve system calls and PIDs for anomaly detection.

By leveraging a ring buffer shared between the kernel and user space, the proposed eBPF-based collection mechanism eliminates the need for data copying and avoids the need for intermediate storage between isolated containers. This design allows the proxy container to access the main container's system calls in real time with minimal overhead, thereby improving collection speed and reducing resource consumption. Moreover, by preserving kernel-level isolation, the monitoring mechanism ensures proxy containers remain protected from potential threats by compromised main containers.

2) Anomaly Detection and Container Isolation: Through the collected system calls and PIDs, the proxy container monitors the main container's behaviors. The anomaly detection and container isolation process consists of a two-stage detection process: the first stage for detecting potentially malicious behavior through system call analysis, and the second stage for verifying these results through request-response traffic comparison. To verify the system call analysis results, the system leverages the inherent horizontal scaling capabilities of cloud-native environments. In Kubernetes environments, applications are typically deployed as multiple identical replicas. By comparing the suspicious container's request-response traffic with other replica containers' traffic, the process can verify the first stage detection results. Once malicious behavior is confirmed, the proposed defense mechanism is activated with traffic control actions (forwarding, relay, or drop), ensuring continued service availability. Additionally, using these verification results, the process collects data for container-specific model training.

Fig. 6 shows the detailed operational flow of container isolation based on the anomaly detection and verification process. The Cluster Information Provider module manages and distributes replica set information, including IP addresses required

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

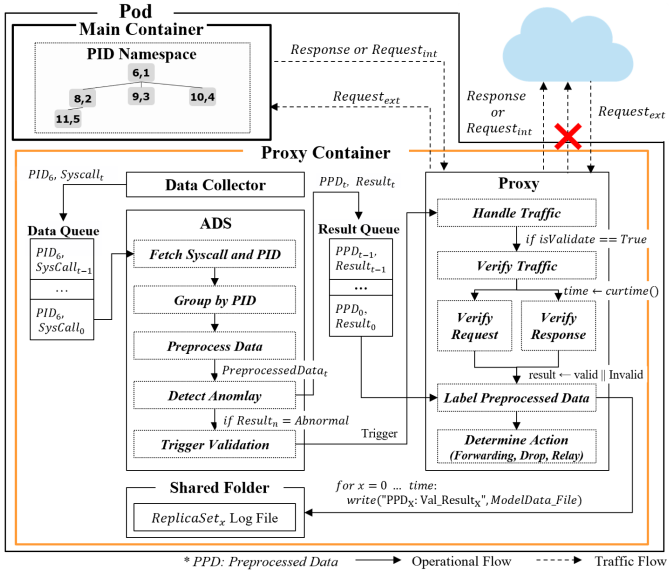


Fig. 6. Operational flow of anomaly detection and container isolation process.

for communication between replicas. Using this replica set information, the Proxy module verifies the anomaly detection results from ADS. When anomalies are detected, the Proxy module maintains service availability by relaying requests to healthy replicas within the replica set instead of compromised containers. Furthermore, the Cluster Information Provider module periodically transmits replica information to the Proxy module. This periodic transmission enables effective adaptation to the dynamic nature of cloud-native environments, where Pods and containers frequently change.

The ADS detects the abnormal behavior of the main container by analyzing system calls collected by the Data Collector module. It maintains a PID-System Call map that uses PID as a key and the corresponding system call sequence as a value. Whenever a system call is collected, the ADS retrieves the corresponding PID and appends the collected system call to the associated sequence. Once the sequence reaches the predefined window size, the anomaly detection model determines whether the sequence exhibits abnormal behavior. Subsequently, the oldest system call is removed from the sequence to maintain the sliding window size. When the sequence is classified as normal, the sequence is logged as normal behavior. Otherwise, the sequence is labeled as abnormal and temporarily stored in a dedicated queue for further verification. The ADS then requests the Proxy module to verify the detection results.

When a verification request is received from the ADS along with sequences of system calls labeled as abnormal, the Proxy module performs two types of verification, leveraging the inherent replicated architecture of cloud-native environments: verification of responses to external requests and verification of internal requests. For external requests, the Proxy module compares the response from the main container with the response from another replica container. Upon detecting a discrepancy that indicates a potential compromise, the Proxy module isolates the potentially compromised main container by relaying subsequent

Algorithm 1: Anomaly Detection Result Verification and Container Isolation Algorithm.

Input: $T \leftarrow$ Traffic, $IsVerify \leftarrow$ Boolean flag indicating whether to perform verification

1 **if** $T.type = Req \wedge IsVerify$ **then**
2 **if** $T.src = Ext$ **then**
   //Response Verification
3 $IP_{replica} \leftarrow$ SelectReplicaIP()
4 $Res_{replica} \leftarrow$ GetResponse($IP_{replica}, T$)
5 **if** $!IsComp$ **then**
6 $Res_{main} \leftarrow$ GetResponse(IP_{main}, T)
7 **if** $Res_{main} \neq Res_{replica}$ **then**
8 $IsComp \leftarrow$ True
9 **end**
10 **end**
11 SendResponse($T.src, Res_{replica}$)
12 **return**
13 **else**
   //Request Verification
14 $Hash \leftarrow$ ComputeHash(T)
15 **if** !VerifyRequest($Hash$) **then**
16 Drop(T)
17 **return**
18 **end**
19 **end**
20 **end**
21 Forward($T.dst, T$)

inbound requests to other replicas. For internal requests, the Proxy module transforms internal traffic into a hash value and transmits the hash value to the Verifier module within the control plane. The Verifier module checks whether the same hash value has been previously transmitted by other replicas within the same replica set. Upon finding a matching record, the traffic is considered normal, whereas traffic without matching records is considered abnormal.

Based on the verification results, the Proxy module relabels the sequences stored in a dedicated queue and logs labeled sequences. This labeled data is used to update the anomaly detection model through an online learning process. The detailed verification process is shown in Algorithm 1 and 2.

In these algorithms, $IsVerify$ indicates whether the verification process should be initiated, while $IsComp$ represents the compromised status of the main container. $P2R$, $R2H$, and $H2I$ are specialized dictionaries used for mapping between Pod IPs (P), replica set IDs (R), hash values (H), and IP sets (I) that sent the hash values. $IsValid$ is a flag indicating whether the internal request is valid.

Algorithm 1 presents the operational details of the process used by the Proxy module to verify both request and response in order to detect potential compromises of the main container. The algorithm consists of two main parts: response verification (lines 2-12) and request verification (lines 13-19).

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Request Verification Algorithm in Control Plane.

Input: $Req \leftarrow$ Verification request from proxy container
Output: $IsValid \leftarrow$ Verification result

$ReplicaSet_ID \leftarrow P2R[Req.src]$
$H2I \leftarrow R2H[ReplicaSet_ID]$
if $Req.Hash \in H2I$ **then**
  $IP_Set \leftarrow H2I[Req.Hash]$
  if $|IP_Set| = 1 \wedge Req.src \in IP_Set$ **then**
    $IsValid \leftarrow$ False
  else
    $IP_Set \leftarrow IP_Set \cup \{Req.src\}$
    $IsValid \leftarrow$ True
  end
else
  $IP_Set \leftarrow \{Req.src\}$
  $IsValid \leftarrow$ False
end
$H2I[Req.Hash] \leftarrow IP_Set$
return $IsValid$

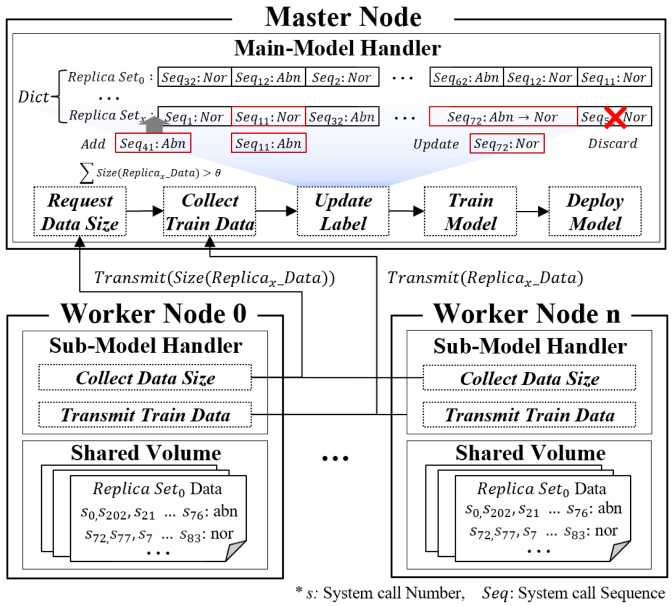


Fig. 7. Operational flow of container-specific online learning process.

Response verification is triggered by external requests. The Proxy module forwards the request to both the main container and a replica Pod. Comparing the responses determines whether the main container behaves consistently with its replicas. When the responses differ, the main container is flagged as potentially compromised. Then, subsequent traffic is redirected to the healthy replicas to maintain service availability.

Request verification focuses on requests originating from the main container. The Proxy module generates a hash value from each outgoing request and sends the hash value to the Verifier module in the control plane. The Verifier module, as shown in Algorithm 2, uses a nested dictionary structure to track which Pods have sent requests with specific hash values. By checking whether other Pods in the same replica set have sent requests with the same hash, the Verifier determines whether the main container's request is consistent with the expected behavior of the replica set. When the hash value is unique to the main container or does not exist, the request is labeled as potentially abnormal. This process helps to ensure that the main container is operating in sync with its replicas and hasn't been compromised.

By combining system call-based anomaly detection with cross-replica traffic verification, this process offers a robust and scalable defense mechanism tailored to microservice environments. Leveraging cloud-native horizontal scaling, it ensures continuous service availability while effectively detecting potential threats through system call analysis and inter-replica traffic pattern comparison.

3) Container-Specific Online Learning: Containerized applications present unique challenges for anomaly detection due to container-specific system call patterns and the need for frequent model updates following application changes. The proposed process addresses these challenges by implementing container-specific online learning, enabling continuous model adaptation with minimal service disruption.

When a new replica set is deployed in a Kubernetes cluster, the proxy container begins with an initial model that treats all system call sequences as anomalous. This conservative approach ensures that no potentially malicious behaviors are initially accepted as normal. Through two-stage detection and container-specific online learning, the model gradually learns and incorporates verified normal sequences into the assigned replica set, building a precise normal behavior profile.

Fig. 7 illustrates the detailed operation of the container-specific online learning process. This process demonstrates how the proposed service mesh system utilizes labeled system call sequences collected from the Proxy module. In a distributed environment, sequence data collected from each replica set is stored across multiple nodes. To process this distributed data, we employ a two-tier handler architecture consisting of a Main-Model Handler module on the master node and Sub-Model Handler modules on worker nodes. The detailed operation of this process is presented in Algorithm 3.

The Main-Model Handler module first requests the number of collected sequences for each replica set from the Sub-Model Handler modules. The Sub-Model Handler modules track and report the number of collected sequences in a logging volume shared with the proxy container. When the number of collected sequences exceeds a predefined threshold θ, the Main-Model Handler module requests the labeled system call sequences from the Sub-Model Handler modules.

For processing collected sequences, the Main-Model Handler module maintains a dictionary data structure mapping each replica set to a fixed-size queue. The dictionary data structure stores sequences and their labels from previous training cycles to progressively define the container's normal behavior patterns. These stored sequences serve as the training dataset that enables

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Container-Specific Online Learning Algorithm.

Input: $\theta \leftarrow$ Sequence threshold, $Q \leftarrow$ Dictionary of queues indexed by ReplicaSet IDs
Output: Updated model per replica set

$ReplicaSets \leftarrow$ GetReplicaSets()
forall $r \in ReplicaSets$ **do**
  $n_r \leftarrow \sum_{h \in \text{SubHandlers}} \text{FetchSequenceCnt}(h, r)$
  if $n_r \geq \theta$ **then**
    $NewSeqs \leftarrow \bigcup_{h \in \text{SubHandlers}} \text{FetchLabeledSeqs}(h, r)$
    forall $(s, label) \in NewSeqs$ **do**
      if $s \in Q[r]$ **then**
        $old_label \leftarrow \text{GetLabel}(Q[r], s)$
        if $label = nor \wedge old_label = abn$ **then**
          UpdateLabel$(Q[r], s, nor)$
        end
      else
        Enqueue$(Q[r], (s, label))$
        if $|Q[r]| > MaxQueueSize$ **then**
          DequeueOldest$(Q[r])$
        end
      end
    end
    $TrainData \leftarrow \text{FilterByLabel}(Q[r], nor)$
    $Model_r \leftarrow \text{TrainModel}(TrainData)$
    DeployModelToProxy$(r, Model_r)$
  end
end

the model to continuously adapt to container-specific behaviors. When the queue reaches its capacity, the oldest sequences are automatically discarded, allowing the model to reflect recent changes in normal behavior due to application updates or environment dynamics.

When processing sequence labels, the Main-Model Handler module updates the labels of sequences stored in the queue corresponding to each replica set based on newly collected sequences and following predefined rules. If identical sequences have conflicting labels between the existing queue and the newly collected data, the normal label takes precedence, updating the abnormal label to normal. This is because sequences verified as normal through the verification process represent genuine normal behavior. Through this approach, sequences previously misclassified as abnormal can be correctly updated to normal. If a sequence is not already present in the queue corresponding to its replica set, the Main-Model Handler module adds the sequence and its label to that queue.

Once the label update process is complete, the Main-Model Handler module trains the model using only the sequences labeled as normal from the queue associated with each replica set. The trained model is then deployed to the proxy containers in the corresponding replica set. After receiving the trained model, the Proxy module replaces its existing model with the newly trained one and resumes system call analysis. By continuously repeating this process, the normal behavior profile becomes increasingly precise and container-specific.

Through this continuous model adaptation, the proposed container-specific online learning effectively addresses the challenges of anomaly detection in dynamic, multi-container environments. Additionally, by employing a distributed handler architecture, the system maintains and updates anomaly detection models across replica sets, consistently ensuring high detection performance even when containers are updated.

V. Security Evaluation

In this section, we analyze the security properties of the proposed service mesh system for real deployments and how it reduces the attack surface. We also evaluate the robustness of its anomaly detection, container isolation, and online learning.

A. Isolation-Preserving System Call Monitoring

Our design adopts the sidecar pattern, placing the main container and the proxy container within a single Pod. The proxy container collects the system calls generated by the main container. These operations require specific privileges and runtime configurations. We discuss these requirements concretely from the perspective of a Kubernetes implementation.

To monitor a main container, the sidecar container requires access to the main container's runtime process information. Traditional approaches obtain this internal process information by either relaxing isolation boundaries or granting additional privileges. In practice, operators commonly enable Pod-level settings such as `shareProcessNamespace: true` or `hostPID: true`, which merge PID namespaces across containers in a Pod or allow the sidecar container to directly access the host's PID namespace with full visibility into all processes.

However, these configurations weaken isolation and expand the attack surface. Enabling `shareProcessNamespace` allows containers that share the same PID namespace to access each other's processes, so a compromise of one container can more easily spread to its peers within the same Pod. Enabling `hostPID` maps the sidecar container's PID namespace to the host's namespace, increasing the risk of container escape or host-level compromise if the sidecar container is breached. Thus, while these measures facilitate system call collecting, they also expose additional attack vectors and introduce significant security risks [27], [45], [46].

To address the privilege issue, the Pod PID Provider of the proposed system supplies the proxy container with the PIDs to be monitored. This set is registered in kernel space as the tracking target, and the PID Tracer module continuously monitors and updates subsequent changes to the main container's PIDs (creation, termination, and restart). As a result, the proxy container does not need to query the main container's internal state, and system call collection proceeds without relaxing container boundaries or relying on the host PID namespace.

B. Robust Anomaly Detection, Isolation and Online Learning

To ensure reliable anomaly detection and maintain label consistency during continuous model updates, the proposed

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

service mesh system incorporates multiple safeguards against false positives and false negatives. Note that adversarial attacks that intentionally disguise malicious behaviors as normal are beyond the scope of this work.

At the initial deployment stage, the system adopts a cautious configuration that treats all sequences as anomalous. Each sequence must pass verification before being classified as normal, ensuring that no potentially malicious behaviors are accepted as normal from the outset.

To achieve this, the system employs a two-stage detection. In the first stage, the anomaly detection model analyzes system call sequences generated by the main container to identify potentially suspicious behaviors, and each detected sequence is flagged for further verification. In the second stage, when abnormal behavior is detected, the system verifies the detection results through a replica-based verification mechanism. This process leverages the horizontal scalability of cloud-native environments by comparing request–response consistency across replicas. Based on the verification, each sequence is labeled as normal or abnormal and stored for subsequent processing.

Building on these verified labels, the online learning procedure selectively updates each replica's model using only verified sequences. Verified normal sequences are retained in a per-replica queue and used for retraining, while verified abnormal sequences are excluded from the learning process. This verification-based filtering prevents the model from incorporating false positives into its decision boundary and ensures that online updates rely solely on confirmed normal behaviors.

However, during ongoing online learning, label conflicts may occasionally arise across replicas. Some sequences labeled as abnormal during verification may actually be normal sequences that appear between truly abnormal ones. When the model misclassifies such normal sequences and groups them with surrounding abnormal sequences during verification, they are collectively labeled as abnormal. To address this, the system prioritizes verified normal labels when inconsistencies arise across replicas, thereby preserving label consistency and preventing the spread of mislabeled data through re-evaluation.

Through these mechanisms, the system ensures that only verified normal behaviors are incorporated into model training. The combination of replica-based verification, conflict resolution, and selective learning maintains model reliability and prevents the incorporation of malicious patterns, ensuring long-term adaptability in evolving cloud-native environments.

VI. Performance Evaluation

The proposed service mesh system raises several practical considerations. This section presents these considerations as research questions and provides experimental results to address them. The key research questions are as follows:

1) **RQ1: What is the overhead of the proposed zero-copy-based system call collection on collection speed and resource consumption?** This question evaluates the efficiency of the proposed system call collection method.
2) **RQ2: Why are container-specific models necessary in cloud-native environments?** This question examines the limitations of unified models in cloud-native environments and assesses the need for container-specific modeling.
3) **RQ3: How do lightweight machine learning models perform in terms of detection accuracy under different conditions in an online learning setting?** This question explores the effectiveness of various models suitable for resource-constrained environments and their adaptability through online learning.
4) **RQ4: How does the detection performance of online learning models compare to that of offline learning models?** This question investigates the benefits of continuous learning in the context of anomaly detection.
5) **RQ5: How does the proposed online learning model adapt to container version updates?** This question evaluates the adaptability of the online learning model when containers undergo version changes.
6) **RQ6: What is the network performance overhead of the proxy component in terms of latency and throughput?** This question assesses the system's impact on network performance, a crucial factor for real-world applicability.

The experiment evaluations were performed on a Kubernetes cluster (v1.28.15) composed of four virtual machines using QEMU and KVM. The cluster consisted of a master node (12 vCPUs, 6 GB RAM) and three worker nodes (3 vCPUs, 6 GB RAM each). All nodes utilized Ubuntu 20.04 LTS with Linux kernel 5.4.0.

A. *Kubernetes-Specific Data Collection Method*

We reviewed existing Intrusion Detection System (IDS) datasets such as ADFA-LD [47], LID-DS [48], and CB-DS [49] to compare the performance of offline and online learning approaches. However, these datasets neither reflect the distributed service environment characteristics of Kubernetes nor contain Kubernetes-related attack scenarios. Therefore, based on the MITRE ATT&CK framework [50], we constructed a dataset comprising six lateral movement attack scenarios. We also collected normal system call data by running three service workloads in Pods deployed within a Kubernetes cluster and generated attack data in the same environment.

1) Attack Data Collection: The dataset comprises six attack scenarios based on MITRE ATT&CK techniques. Three scenarios specifically target vulnerabilities and behaviors unique to Kubernetes environments, while the remaining three represent general attack strategies applicable to traditional network infrastructures. Each scenario assumes that an adversary has already gained initial access to a compromised container within the cluster. Table II summarizes the system call data collected during the execution of each attack scenario. Each technique ID (e.g., T1528) corresponds to a specific adversarial technique defined in the MITRE ATT&CK framework. The details of each scenario are as follows:

- *To enumerate Kubernetes cluster information:* The adversary first performs directory traversal to collect container-level information (T1589). Subsequently, the adversary obtains API credentials that grant access to the Kubernetes

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ATTACK DATA COLLECTION RESULTS FOR NETWORK-BASED LATERAL MOVEMENT ATTACK SCENARIOS: THREE KUBERNETES-SPECIFIC AND THREE GENERAL CASES

Category	Attack Scenario	Techniques#	Syscall#
K8s-specific Network Attack	Enumerate K8s cluster information	T1528, T1589, T1613	93,519
	Manipulate K8s Resources	T1528, T1589, T1613, T1609	95,436
	Escape from Container to Host System	T1528, T1589, T1610, T1611	121,402
General Network Attack	Spoof Web Content	T1565	94,055
	Brute Force Attack	T1595, T1110	338,063
	Remote Access	T1219	72,681

TABLE III
NORMAL DATA COLLECTION RESULTS FOR WORKLOAD SCENARIOS: FOUR WEB-BASED, ONE CI/CD, AND TWO MONITORING CASES

Workload	Tools	Function	Syscall#
Web	React, Nginx	UI/UX, Proxy	54,805
	FastAPI, Guvicorn	Python Core	1,086,626
	PostgreSQL	Database	241,412
	pgAdmin	DB management	2,463
CI/CD	Jenkins	CI/CD	290,260
Monitoring	Prometheus	Collection	253,629
	Grafana	Visualization	187,817

API server (T1528) and uses them to enumerate containers, network policies, and other cluster resources.

- *To manipulate Kubernetes resources:* Similar to the previous scenario, the adversary exploits API credentials to gather information. Using the granted API permissions, the adversary modifies cluster resources (T1609), altering network policies to expand their network access range.
- *To escape from container to host system:* As described in Section III, the adversary leverages API credentials to request the API server to create a backdoor container with elevated privileges (T1610). The adversary then accesses the backdoor container and attempts to escape to the host system using these elevated privileges (T1611).
- *To conduct web content spoofing:* The adversary infiltrates an Nginx web server container and modifies the Nginx configuration file to return manipulated content when web pages are requested (T1565).
- *To execute brute force attack:* The adversary performs network scanning within the cluster from the compromised container (T1595). Upon identifying remote access services, the adversary attempts to obtain service passwords through brute force attacks (T1110)
- *To install remote access software:* The adversary installs persistence-enabling software such as tmate on the compromised container to maintain persistent access (T1219). This persistent remote access allows continued malicious activities even after system reboots or network changes.

2) Normal Data Collection: The normal workloads included an online survey web service [51], continuous integration/deployment (CI/CD) pipelines [52], and monitoring services. Each service was deployed as one or more Pods in the Kubernetes cluster to collect system calls. The collection results are summarized in Table III, and the configuration details of each service are as follows:

- *Web service:* This service is an online survey application [51] consisting of a React frontend served through Nginx and a FastAPI backend running on a gunicorn. It uses PostgreSQL as the database with pgAdmin for management. Each component is deployed as a separate Pod.
- *CI/CD service:* Jenkins is used to implement the CI/CD pipeline. The Jenkins master dynamically spawns agent Pods that clone the GitHub repository, build container images, push them to Docker Hub, and deploy applications via the Kubernetes API server.
- *Cluster monitoring service:* Cluster-wide monitoring is performed using Prometheus and Grafana. Prometheus serves as a time-series database collecting metrics from components within the cluster, while Grafana visualizes the collected data through dashboards.

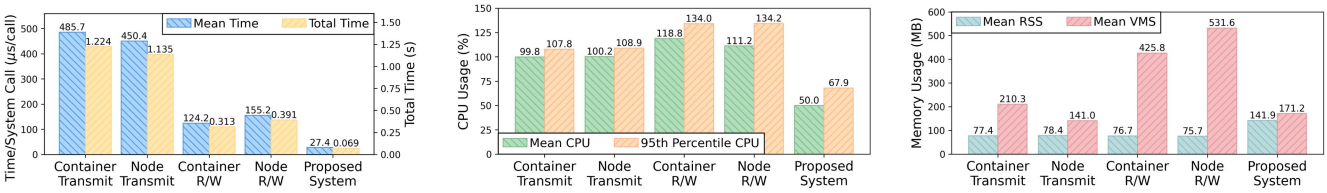
B. RQ1: Performance Analysis of Zero-Copy-Based System Call Collection Mechanism

To evaluate the effectiveness of the proposed zero-copy–based system call collection method, we compare it against four practical approaches derived from the process-visibility constraint of Linux PID namespaces. Because target processes in the main container can be identified only from within the container itself or from the host node, our baselines vary along these two initiation points.

As illustrated in Fig. 9, system call collection can be initiated either from the main container or from the host node. In both cases, events are delivered to the proxy container either by direct communication or by writing to and reading from a host-mounted directory. In this process, the proxy container responsible for system call collection either opens an endpoint to receive system calls or reads from mounted folders. For communication between system call collection points and the proxy container, we utilized the Python `aiohttp` library for transport, while file read/write operations use a Kubernetes hostPath mount in the Pod configuration. Moreover, as the system call collection tool in the comparative approaches for baselines, we used `strace`, a Linux diagnostic and debugging utility that monitors interactions between processes and the Linux kernel, including system calls.

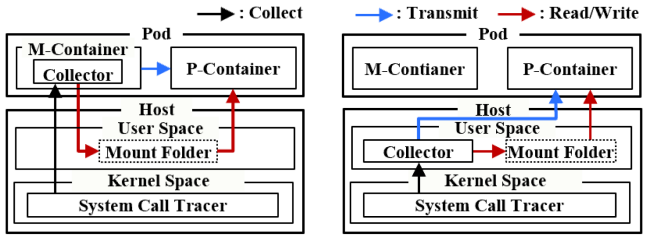
In contrast to these paths, which incur data copies during file writes and message transfers, the proposed zero-copy design maps the kernel's ring buffer into the proxy process and reads events directly from memory. This avoids per-event copying in the data path, reduces I/O overhead, and enables faster collection with lower CPU utilization in our evaluation.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.



(a) System call collection latency comparison (b) CPU utilization during collection (c) Memory utilization during collection

Fig. 8. Performance comparison of system call collection approaches. (a) System call collection latency comparison. The proposed system demonstrates significantly faster collection speed compared to other methods; (b) CPU usage comparison during system call collection. The proposed system exhibits approximately half the CPU usage compared to other methods; (c) Memory usage comparison during system call collection.



(a) System call collection initiated from the main container (b) System call collection initiated from the host node

Fig. 9. System call collection architectures evaluated. In both architectures, events are delivered to the proxy container either via direct messaging or via a host-mounted directory.

1) Latency Overhead: Fig. 8(a) presents the system call collection latency results, measured while generating 2,520 system calls in the main container. The proposed system demonstrates the fastest collection speed among the comparison groups, with a total of 0.069 s and an average of 27.4 μs per system call. Among the comparison groups, the method of writing to and reading from a host-mounted folder in the container, which showed the fastest collection speed, performs at a total of 0.313 s and an average of 124.2 μs per system call. This represents an approximately 4.5 times difference to the proposed system, confirming the superior collection speed of the zero-copy-based system call collection method.

2) Resource Overhead: To compare resource overhead, we measured CPU and memory usage during the system call collection process over a three-minute period. For this performance evaluation, we used the Python `psutil` library, which enables system and process management tasks and can monitor major system resource usage, including CPU, memory, disk, and network.

As shown in Fig. 8(b), for CPU overhead, the proposed system shows an average of 50% and a 95th percentile of 67.9%. The comparison groups show usage above 100% for both the average and 95th percentile, indicating approximately twice the CPU usage. The performance difference occurs because the zero-copy-based method collects data through a single process in the proxy container, whereas comparison methods involve collecting data from the main container or host and subsequently receiving it in the proxy container.

Regarding memory utilization, Fig. 8(c) compares resource usage by measuring RSS (Resident Set Size) and VMS (Virtual Memory Size). RSS represents the memory size allocated to the process, while VMS represents the accessible memory size, including allocated but unused memory. The proposed system used an average RSS of 141.9 MB over three minutes. Among the comparison groups, the method of writing to a mounted folder on the host and reading from the proxy container showed the lowest RSS performance, with a usage of approximately 75.7 MB. This shows a two-fold difference compared to the proposed system, which is attributed to its use of a ring buffer for system call transmission between the eBPF-based kernel process and the collecting process.

C. RQ2: Unified Vs. Container-Specific Models

To validate the necessity of container-specific models in cloud-native environments, we compared a unified anomaly detection model against container-specific anomaly detection models on our Kubernetes-specific dataset. We preprocessed the system call sequences using a sliding window of length 40, applied uniformly across all models. The unified model was trained by uniformly sampling normal data to balance classes from all services, whereas each container-specific model was trained only on the data of its corresponding service. We evaluated seven commonly used ML methods—isolation forest (IF), k-means, local outlier factor (LOF), one-class support vector machine (OC-SVM), principal component analysis (PCA), elliptic envelope, and gaussian mixture.

Table IV shows that container-specific models achieve higher ROC-AUC for every method across all services. On average across methods, the unified models reach a mean ROC-AUC of 0.436, while the container-specific models reach 0.722, for an average absolute gain of 0.286. The ROC-AUC improvement ranges from 0.169 for PCA to 0.373 for OC-SVM across methods. Precision and recall also increase on average by 0.130 and 0.081, respectively, indicating reductions in both false positives and false negatives. The best-performing method varies by service: LOF for pgAdmin, Grafana, and Prometheus; IF for Nginx+React and Postgres; and OC-SVM for Jenkins. Unified models show lower ROC-AUC overall and, for some methods such as IF and Elliptic Envelope in particular, high recall with low precision.

These results indicate that system call feature distributions differ across services in practice. This heterogeneity makes it difficult to define a single common normal distribution at scale and motivates container-specific modeling. In our experiments, per-service training consistently improves ROC-AUC over a

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
UNIFIED VERSUS CONTAINER-SPECIFIC ANOMALY DETECTION MODELS ON THE KUBERNETES-SPECIFIC DATASET. THE UNIFIED MODEL IS TRAINED ON NORMAL DATA FROM ALL SERVICES; EACH CONTAINER-SPECIFIC MODEL IS TRAINED ONLY ON ITS TARGET SERVICE.

Method	Metric	Unified Model	Container-Specific Model						
			Nginx+React	FastAPI	pgAdmin	Postgres	Jenkins	Prometheus	Grafana
Isolation Forest (IF)	Precision	0.519	0.846	0.768	0.769	0.877	0.764	0.530	0.655
	Recall	0.892	0.952	0.938	0.720	0.981	0.781	0.971	0.544
	ROC-AUC	0.491	0.945	0.907	0.684	0.913	0.850	0.507	0.678
k-means	Precision	0.656	0.592	0.810	0.776	0.851	0.691	0.619	0.728
	Recall	0.427	0.987	0.974	0.720	0.986	0.939	0.199	0.648
	ROC-AUC	0.415	0.588	0.924	0.660	0.899	0.680	0.524	0.709
Local Outlier Factor (LOF)	Precision	0.560	0.603	0.870	0.995	0.860	0.688	0.761	0.719
	Recall	0.572	0.448	0.901	0.852	0.491	0.698	0.484	0.593
	ROC-AUC	0.461	0.515	0.845	0.948	0.664	0.829	0.714	0.733
One-Class Support Vector Machine (OC-SVM)	Precision	0.593	0.644	0.830	0.781	0.742	0.849	0.608	0.603
	Recall	0.930	0.991	0.882	0.996	0.990	0.761	0.921	0.977
	ROC-AUC	0.373	0.556	0.916	0.723	0.804	0.901	0.604	0.718
Principal Component Analysis (PCA)	Precision	0.514	0.520	0.569	0.707	0.607	0.820	0.568	0.503
	Recall	0.923	0.966	0.816	0.881	0.844	0.781	0.942	0.659
	ROC-AUC	0.488	0.509	0.620	0.807	0.678	0.859	0.622	0.503
Elliptic Envelope	Precision	0.598	0.910	0.697	0.615	0.856	0.555	0.607	0.626
	Recall	0.925	0.821	0.961	0.407	0.961	0.947	0.933	0.929
	ROC-AUC	0.409	0.938	0.761	0.566	0.906	0.429	0.563	0.719
Gaussian Mixture	Precision	0.656	0.592	0.810	0.776	0.851	0.691	0.619	0.728
	Recall	0.427	0.987	0.974	0.720	0.986	0.939	0.199	0.648
	ROC-AUC	0.415	0.588	0.924	0.660	0.899	0.680	0.524	0.709

unified model. Accordingly, we adopt container-specific models for robustness and propose an online learning strategy to train them efficiently in deployment.

D. RQ3: Detection Performance of Machine Learning Models Under Different Online Learning Parameters

To analyze the performance between ML models during online learning, we measured changes in model performance by iteratively deploying models trained on data collected through the proposed data collection process. In this experiment, we used the FastAPI dataset with a window size of 40 system calls. For accurate performance comparison between models, we deployed a FastAPI container as the main container and multiple proxy containers in a Pod, ensuring they collect identical system calls and perform anomaly detection at the same time. Through this setup, except for round 0, where proxy containers have identical models, each container learns different system call sequence datasets based on its model's detection results. This online learning process was performed over six rounds, excluding the initial model, comparing the seven ML methods previously assessed in RQ2.

Based on the experimental results shown in Table V, we observed varying performance patterns across different machine learning models during the online learning process. Gaussian Mixture demonstrated the most consistent and superior performance overall, showing steady improvement in ROC-AUC scores from 0.894 to 0.918 across six rounds while achieving high precision (between 0.875 and 0.931) and recall (between 0.752 and 0.851). K-means also exhibited stable performance with consistently high precision (around 0.93) and gradually improved ROC-AUC scores (from 0.894 to 0.911). LOF showed significant improvement over the rounds, particularly in recall, which increased from 0.570 to 0.872, and ROC-AUC, which improved from 0.738 to 0.911. However, IF, OC-SVM, PCA, and EllipticEnvelope demonstrated relatively lower and more unstable performance. PCA's ROC-AUC scores fluctuated significantly, ranging from 0.659 to 0.885, while EllipticEnvelope maintained consistently lower performance metrics compared to other models, with ROC-AUC scores between 0.700 and 0.754. These results suggest that Gaussian Mixture and K-means are more suitable for online learning in our system, particularly for detecting anomalies in containerized environments.

E. RQ4: Comparison With Online and Offline Anomaly Detection Methods

Comparative experiments were conducted to evaluate the performance difference between online and offline learning approaches. For this evaluation, we used two different datasets with a window size of 40 system calls: our Kubernetes-specific FastAPI dataset and the ADFA-LD dataset [47]. These experiments compared the online learning model, presented in Section VI-D, with offline learning models proposed in previous studies [22], [23], [24], [25]. Unlike the online learning model, which learned normal data derived from detection and verification, the offline learning model was trained only on normal data from the first three rounds out of six rounds. For the CNN-based model [23], normal data from the first three rounds was used for training in the same way as the other offline models, but as it employs a supervised learning approach, it also utilized 70% of the attack data during training.

1) Performance Evaluation Using Kubernetes-Specific Dataset: Table VI presents the comparative experimental

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF PRECISION, RECALL, AND ROC-AUC SCORES FOR SEVEN MACHINE LEARNING MODELS ACROSS SIX ROUNDS OF ONLINE LEARNING. THE TABLE HIGHLIGHTS THE VARYING PERFORMANCE PATTERNS OF DIFFERENT METHODS, WITH GAUSSIAN MIXTURE AND K-MEANS SHOWING CONSISTENTLY STRONG AND IMPROVING PERFORMANCE.

Method	Metric	Online Learning Round					
		1	2	3	4	5	6
Isolation Forest (IF)	Precision	0.848	0.833	0.827	0.839	0.835	**0.854**
	Recall	**0.739**	0.696	0.712	0.688	0.685	0.670
	ROC-AUC	**0.878**	0.853	0.853	0.839	0.832	0.823
k-means	Precision	0.934	**0.936**	0.933	0.934	**0.936**	0.935
	Recall	0.752	0.748	0.760	0.765	**0.766**	**0.766**
	ROC-AUC	0.894	0.893	0.901	0.907	**0.912**	0.911
One-Class Support Vector Machine (OC-SVM)	Precision	0.904	0.826	0.825	0.898	**0.907**	0.903
	Recall	0.649	0.779	**0.833**	0.645	0.637	0.637
	ROC-AUC	0.829	0.873	**0.894**	0.807	0.800	0.795
Principal Component Analysis (PCA)	Precision	0.806	0.815	0.706	0.805	**0.848**	0.832
	Recall	0.516	0.515	0.576	0.487	**0.762**	0.443
	ROC-AUC	0.716	0.705	0.718	0.704	**0.885**	0.659
EllipticEnvelope	Precision	0.847	0.842	0.845	0.836	**0.868**	0.819
	Recall	0.589	0.582	0.572	0.568	0.552	**0.621**
	ROC-AUC	0.742	0.716	0.709	0.700	0.725	**0.754**
Gaussian Mixture	Precision	**0.931**	**0.931**	0.873	0.872	0.872	0.875
	Recall	0.752	0.768	0.844	0.847	0.846	**0.851**
	ROC-AUC	0.894	0.911	0.915	0.915	0.915	**0.918**
Local Outlier Factor (LOF)	Precision	0.877	0.834	0.847	0.828	0.855	**0.860**
	Recall	0.570	0.816	0.820	0.867	**0.872**	0.849
	ROC-AUC	0.738	0.878	0.881	0.888	0.909	**0.911**

TABLE VI
COMPARISON OF THE PROPOSED ONLINE LEARNING MODEL (LOF, ROUND 6) AGAINST EXISTING OFFLINE MODELS USING THE KUBERNETES-SPECIFIC DATASET

Method	Precision	Recall	ROC-AUC
CNN [23]	0.903	0.699	0.903
BiLSTM-VAE [24]	**0.924**	0.608	0.907
DeepLog [22]	0.494	**0.975**	0.311
Zero-SAD [25]	0.625	0.967	0.479
LOF (R6)	0.860	0.849	**0.911**

TABLE VII
COMPARISON OF THE PROPOSED ONLINE LEARNING MODEL (LOF, ROUND 6) AGAINST EXISTING OFFLINE MODELS USING THE ADFA-LD DATASET

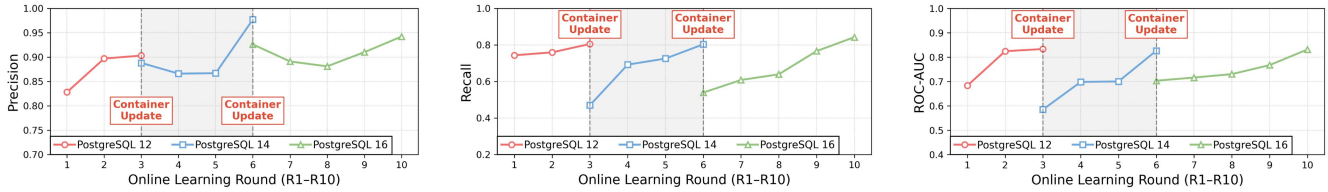
Method	Precision	Recall	ROC-AUC
CNN [23]	0.954	0.627	0.951
BiLSTM-VAE [24]	**0.987**	0.731	0.769
DeepLog [22]	0.500	**0.989**	0.654
Zero-SAD [25]	0.729	0.977	0.938
LOF (R6)	**0.987**	0.920	**0.980**

results using the Kubernetes-specific dataset. The experimental results show that the online learning model (LOF at round 6) achieved balanced and improved performance across all metrics. CNN [23] exhibited notable precision and ROC-AUC, both at 0.903, but had a relatively limited recall at 0.699. BiLSTM-VAE [24] showed a high precision of 0.924 but a relatively low recall of 0.608. In contrast, DeepLog [22] and Zero-SAD [25] showed high recall of 0.975 and 0.967 but low precision of 0.494 and 0.625, respectively. The online learning model maintained consistently high performance with 0.860 precision and 0.849 recall. In particular, our model achieved the highest ROC-AUC score of 0.911 compared to the other offline models. This shows that online learning provides more reliable and balanced anomaly detection capabilities over time compared to traditional offline learning methods.

2) Performance Evaluation Using ADFA-LD Dataset: In addition to the Kubernetes-specific dataset, we evaluated our models using ADFA-LD [53]. Since the ADFA-LD dataset is system call data collected from Ubuntu 11.04, system call numbers were converted to match the system call mappings of Ubuntu 20.04 as used in the experimental environment.

For evaluation, we used the ADFA-LD dataset with the LOF model trained on our Kubernetes-specific data. The LOF model showed steady improvement over rounds in the online learning process, with its AUC score rising from 0.906 in round 1 to 0.981 in round 6. This progression indicates that the model gradually adapts to the behavior patterns of the target container. Table VII presents the comparative analysis results. The online learning model using LOF in round 6 outperformed all offline models across every metric. CNN recorded a strong precision of 0.954 but had a limited recall of 0.627. BiLSTM-VAE produced an even higher precision of 0.987, but its recall remained modest at 0.731. DeepLog exhibited a high recall of 0.989 but a low precision of 0.500. Although Zero-SAD showed a balanced performance with a precision of 0.729 and a recall of 0.977, LOF outperformed it with a precision of 0.987 and a recall of 0.920. Notably, the online learning model achieved the highest

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.



(a) Precision variation across model updates (b) Recall variation across model updates (c) ROC-AUC variation across model updates

Fig. 10. Adaptation of the proposed online learning model to PostgreSQL container version updates. (a) Precision, (b) Recall, and (c) ROC-AUC metrics over ten online learning rounds. Container updates (vertical dashed lines) indicate version transitions (v12→ v14→ v16), during which temporary performance degradation occurs due to behavioral shifts, followed by recovery through continuous model updates.

ROC-AUC score of 0.980, surpassing all other offline models. These results further confirm that our online learning approach maintains consistent and balanced performance across different datasets.

F. RQ5: Model Adaptation Under Container Version Changes

To evaluate the effect of container updates on the proposed online learning model, we measured anomaly detection performance across different PostgreSQL [54] versions (12-alpine, 14-alpine, and 16-alpine). The PostgreSQL container was employed as the database for an online survey application [51], configured to collect system call traces generated during actual web service operations. The experiment consisted of ten online learning rounds in total. Specifically, three model updates were performed on PostgreSQL 12-alpine, followed by three additional updates on PostgreSQL 14-alpine using the round-3 model as the initial model, and four further updates on PostgreSQL 16-alpine using the round-6 model as the initial model. The results are presented in Fig. 10.

The model performance consistently improved through repeated updates within each container version. However, when the container was upgraded to a newer version, the previously trained model experienced a temporary degradation due to unlearned behavioral patterns introduced by the new environment. Precision remained relatively stable since the majority of benign behaviors remained consistent across versions, while recall temporarily dropped as the model initially misclassified previously unseen normal behaviors as anomalies. This indicates that the model retains previously learned knowledge but is sensitive to distributional shifts caused by software or configuration changes. After several online learning rounds, the model gradually incorporates new behavioral patterns, restoring balanced performance without requiring offline learning. Such behavior suggests that the proposed method can maintain long-term adaptability by gradually learning new behavioral patterns while preserving previously acquired knowledge, even when the operating environment changes.

Overall, these findings demonstrate that the proposed online learning approach can gradually recover and maintain stable detection performance after container updates. This confirms the model's adaptability and resilience to version changes commonly encountered in cloud-native environments.

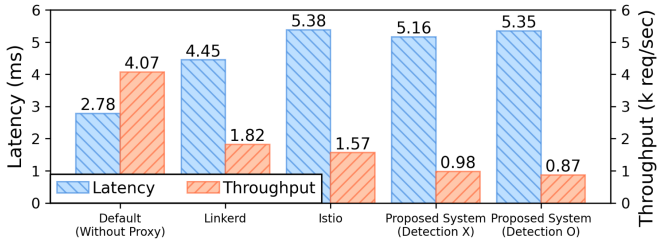


Fig. 11. Comparison of latency and throughput between the default configuration, Linkerd, Istio, and the proposed service mesh system (with and without anomaly detection).

G. RQ6: Network Performance Overhead

We evaluated the network overhead of the proposed service mesh system by measuring latency and throughput using the wrk and wrk2 benchmarks. The evaluation compared our system's performance against the default configuration (direct connection without proxy) to measure baseline performance. We also conducted benchmarking against established Service Mesh solutions, Linkerd [10] and Istio [9], to evaluate performance relative to industry-standard tools. In this experiment, Nginx served as the main container for these measurements.

Fig. 11 presents the results for the following configurations: a default configuration without a proxy, Linkerd, Istio, the proposed system with forwarding traffic only, and the proposed system with both traffic forwarding and anomaly detection. For this evaluation, Linkerd and Istio were used with their default configurations, excluding optional security features like mTLS or authentication/authorization.

When utilizing a proxy, Linkerd exhibited a latency of 4.45 ms and a throughput of 1.82 k req/sec, while Istio showed a latency of 5.38 ms and a throughput of 1.57 k req/sec. The proposed system demonstrated latencies of 5.16 ms and 5.35 ms for the cases with and without anomaly detection, respectively, which are comparable to Istio's. In terms of throughput, the proposed system showed lower performance compared to the other systems, with approximately a 37.58% and 45.58% decrease compared to Istio without and with detection, respectively. Finally, comparing the proposed system with and without anomaly detection revealed that performing anomaly detection resulted in a minimal performance impact, with only a 3.55% increase in latency and an 11.22% decrease in throughput.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

These results indicate that the proposed system can efficiently process traffic with minimal performance overhead, even when concurrently executing machine learning models for anomaly detection within the proxy container.

VII. Discussion

This study evaluated the proposed zero-copy-based system call collection method in terms of efficiency and resource overhead, as well as the necessity, adaptability, and stability of the container-specific online learning mechanism in cloud-native environments. The experiments were conducted on a single Kubernetes cluster using representative workloads to assess practical applicability. While evaluations on larger clusters or different orchestration platforms were not performed, the results demonstrate that the proposed architecture remains effective and scalable in realistic cluster conditions.

As future work, we plan to systematically extend the evaluation to larger and more heterogeneous clusters to validate scalability and applicability under diverse workload patterns. Further research will focus on two key directions: (i) developing adaptive resource allocation strategies that dynamically adjust the ring buffer size and proxy resource usage based on per-container behaviors, and (ii) investigating and formalizing model rollback and version management to automatically recover from detection drift or performance degradation during online learning. These enhancements will significantly improve the stability and efficiency of large-scale deployments.

Finally, to address the data scarcity issue in cloud-native anomaly detection and to promote reproducibility, we publicly release the Kubernetes-specific dataset collected from real cluster environments and the complete source [55].

VIII. Conclusion

This study presents a service mesh-based system designed to detect and mitigate lateral movement attacks in Kubernetes environments. System call collection experiments demonstrate that the proposed approach improves collection speed by up to 4.5 times and reduces CPU usage by nearly half compared to existing methods. Through empirical evaluation, we confirmed the necessity of container-specific models in cloud-native environments and demonstrated that the proposed online learning approach enables continuous adaptation to each container's behavioral patterns, progressively enhancing detection accuracy over time. Moreover, online learning models exhibit superior detection performance compared to offline counterparts across both Kubernetes-specific and standard datasets. The system demonstrates practical applicability through its full implementation within Kubernetes without requiring kernel modifications or additional infrastructure, ensuring portability and seamless integration into existing environments.

References

[1] D. Gannon, R. Barga, and N. Sundaresan, "Cloud-native applications," *IEEE Cloud Comput.*, vol. 4, no. 5, pp. 16–21, Sep./Oct. 2017.

[2] S. Sultan, I. Ahmad, and T. Dimitriou, "Container security: Issues, challenges, and the road ahead," *IEEE Access*, vol. 7, pp. 52976–52996, 2019.

[3] Y. Yang, W. Shen, B. Ruan, W. Liu, and K. Ren, "Security challenges in the container cloud," in *Proc. 3rd IEEE Int. Conf. Trust, Privacy Secur. Intell. Syst. Appl.*, 2021, pp. 137–145.

[4] E. Casalicchio and S. Iannucci, "The state-of-the-art in container technologies: Application, orchestration and security," *Concurrency Computation, Pract. Experience*, vol. 32, no. 17, 2020, Art. no. e5668.

[5] MITRE, "Lateral movement," 2024. [Online]. Available: https://attack.mitre.org/tactics/TA0008/

[6] A. Z. J. Chen and A. Sasson, "Hildegard: New teamtnt cryptojacking malware targeting kubernetes," 2024. [Online]. Available: https://unit42.paloaltonetworks.com/hildegard-malware-teamtnt/

[7] CNCF, "Service mesh," 2024. [Online]. Available: https://landscape.cncf.io/

[8] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han, "Service mesh: Challenges, state of the art, and future research opportunities," in *Proc. IEEE Int. Conf. Service-Oriented Syst. Eng.*, 2019, pp. 122–1225.

[9] istio, "istio," 2024. [Online]. Available: https://istio.io/

[10] linkerd, "linkerd," 2024. [Online]. Available: https://linkerd.io/

[11] "The state of kubernetes security report: 2024 edition," 2024. [Online]. Available: https://www.redhat.com/en/engage/state-kubernetes-security-report-2024/

[12] kubernetes, "kubernetes," 2024. [Online]. Available: https://kubernetes.io/

[13] S. Lee and J. Nam, "KUNERVA: Automated network policy discovery framework for containers," *IEEE Access*, vol. 11, pp. 95616–95631, 2023.

[14] S. Ghavamnia, T. Palit, A. Benameur, and M. Polychronakis, "Confine: Automated system call policy generation for container attack surface reduction," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses*, San Sebastian, Spain, Oct. 2020, pp. 443–458. [Online]. Available: https://www.usenix.org/conference/raid2020/presentation/ghavanmnia

[15] Y. Li, C. Jia, X. Hu, and J. Li, "Kano: Efficient container network policy verification," in *Proc. IEEE Symp. High- Perform. Interconnects*, 2020, pp. 63–70.

[16] H. Zhu, C. Gehrmann, and P. Roth, "Access security policy generation for containers as a cloud service," *SN Comput. Sci.*, vol. 4, no. 6, 2023, Art. no. 748.

[17] A. K. Bhardwaj, P. Dutta, and P. Chintale, "AI-powered anomaly detection for kubernetes security: A systematic approach to identifying threats," *Babylonian J. Mach. Learn.*, vol. 2024, pp. 142–148, 2024.

[18] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "Xi commandments of kubernetes security: A systematization of knowledge related to kubernetes security practices," in *Proc. IEEE Secure Develop.*, 2020, pp. 58–64.

[19] J. A. Curtis and N. U. Eisty, "The kubernetes security landscape: AI-driven insights from developer discussions," in *Proc. IEEE/ACIS 23rd Int. Conf. Soft. Eng. Res., Manage. Appl.*, 2025, pp. 179–186.

[20] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source kubernetes manifests: An empirical study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, pp. 1–36, 2023.

[21] Y. Wang et al., "ContainerGuard: A real-time attack detection system in container-based Big Data platform," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3327–3336, May 2022.

[22] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2017, pp. 1285–1298, doi: 10.1145/3133956.3134015.

[23] H. Chen, R. Xiao, and S. Jin, "Real-time detection of cloud tenant malicious behavior based on CNN," in *Proc. IEEE Intl. Conf. Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Soc. Comput. Netw.*, 2020, pp. 998–1005.

[24] Y. Wang, X. Chen, Q. Wang, R. Yang, and B. Xin, "Unsupervised anomaly detection for container cloud via BiLSTM-based variational auto-encoder," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2022, pp. 3024–3028.

[25] J.-S. Kim, J. Seo, S.-J. Hwang, J. Shin, and Y.-H. Choi, "Zero-sad: Zero-shot learning using synthetic abnormal data for abnormal behavior detection on private cloud," in *Proc. ACM Symp. Cloud Comput.*, New York, NY, USA, 2024, pp. 111–125, doi: 10.1145/3698038.3698533.

[26] AWS, "Aws app mesh," 2025. [Online]. Available: https://aws.amazon.com/app-mesh/

[27] S. Suneja, A. Kanso, and C. Isci, "Can container fusion be securely achieved," in *Proc. 5th Int. Workshop Container Technol. Container Clouds*, 2019, pp. 31–36.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.

[28] D. Zhan, K. Tan, L. Ye, H. Yu, and H. Liu, "Container introspection: Using external management containers to monitor containers in cloud computing," *Comput., Mater. Continua*, vol. 69, no. 3, pp. 3783–3794, 2021.
[29] F. Davidović, "Automating security hardening for containers," Master's thesis, Eindhoven Univ. Technol., Eindhoven, The Netherlands, 2023.
[30] G. Ho et al., "Hopper: Modeling and detecting lateral movement," in *Proc. 30th USENIX Secur. Symp. Assoc.*, Aug. 2021, pp. 3093–3110. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/ho
[31] J. Khoury, H. Klisura, G. D. L. T. Zanddizari, P. Parra Najafirad, and E. Bou-Harb, "Jbeil: Temporal graph-based inductive learning to infer lateral movement in evolving enterprise networks," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3644–3660.
[32] Z. Tian et al., "Real-time lateral movement detection based on evidence reasoning network for edge computing environment," *IEEE Trans. Ind. Informat.*, vol. 15, no. 7, pp. 4285–4294, Jul. 2019.
[33] V. Ramamoorthi, "Anomaly detection and automated mitigation for microservices security with AI," *Appl. Res. Artif. Intell. Cloud Comput.*, vol. 7, no. 6, pp. 211–222, 2024.
[34] ebpf, 2024. [Online]. Available: https://ebpf.io/
[35] M. Desnoyers, "Using the linux kernel tracepoints," 2024. [Online]. Available: https://docs.kernel.org/trace/tracepoints.html
[36] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," *ACM Trans. Privacy Secur.*, vol. 26, no. 3, Jun. 2023, pp. 1–36, doi: 10.1145/3588771.
[37] J. Zengy et al., "SHADEWATCHER: Recommendation-guided cyber threat analysis using system audit records," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 489–506.
[38] B. Bowman, C. Laprade, Y. Ji, and H. H. Huang, "Detecting lateral movement in enterprise computer networks with unsupervised graph AI," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses*, San Sebastian, Spain, Oct. 2020, pp. 257–268. [Online]. Available: https://www.usenix.org/conference/raid2020/presentation/bowman
[39] S. Guo et al., "A zero-day container attack detection based on ensemble machine learning," in *Proc. IEEE 28th Int. Conf. Emerg. Technol. Factory Automat.*, 2023, pp. 1–8.
[40] NIST, "CVE-2024-46257," 2024. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2024-46257
[41] NIST, "CVE-2024-49368," 2024. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2024-49368
[42] NIST, "CVE-2019-11043," 2019. [Online]. Available: https://nvd.nist.gov/vuln/detail/cve-2019-11043
[43] BPF map, 2024. [Online]. Available: https://docs.kernel.org/bpf/maps.html
[44] T. L. K. Archives, "BPF ring buffer," 2024. [Online]. Available: https://docs.kernel.org/bpf/ringbuf.html
[45] Kubernetes, "Share process namespace between containers in a pod," 2025. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/share-process-namespace/
[46] A. Halinen, "Security risks for sidecar containers in kubernetes," 2024.
[47] G. Creech and J. Hu, "A semantic approach to host-based intrusion detection systems using contiguousand discontiguous system call patterns," *IEEE Trans. Comput.*, vol. 63, no. 4, pp. 807–819, Apr. 2014.
[48] M. Grimmer, M. M. Röhling, D. Kreusel, and S. Ganz, "A modern and sophisticated host based intrusion detection data set," *IT Sicherheit als Voraussetzung für eine erfolgreiche Digitalisierung*, vol. 11, pp. 135–145, 2019.
[49] A. Halinen, "Security risks for sidecar containers in kubernetes," Master's thesis, Aalto University School of Science, Espoo, Finland, 2023.
[50] MITRE, "MITRE ATT&CK," 2024. [Online]. Available: https://attack.mitre.org/
[51] SAVE, "SAVE," 2025. [Online]. Available: https://etudeworld.net/
[52] R. Hat, "What is CI/CD," 2024. [Online]. Available: https://www.redhat.com/en/topics/devops/what-is-ci-cd
[53] Sydney, "ADFA IDS datasets," 2024. [Online]. Available: https://research.unsw.edu.au/projects/adfa-ids-datasets
[54] PostgreSQL, "Postgresql," 2024. [Online]. Available: https://www.postgresql.org/
[55] G. Yoon, "Container-specific service mesh-based system for mitigating lateral movement attacks," 2026. [Online]. Available: https://github.com/Yoongunwo/Container-Specific_Service_Mesh-based_System_for_Mitigating_Lateral_Movement_Attacks



Geonwoo Yoon was born in Daegu, South Korea in 1999. He received the BE degree, in 2024 from Pusan National University, Busan, South Korea, where he is currently working toward the MS degree in computer science and engineering. His research focuses on monitoring, anomaly detection, and security in containerized and cloud-native environments.



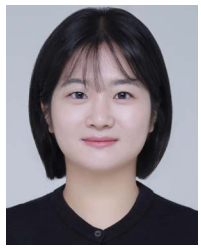
Jinmyeong Shin received the BS and PhD degrees from Pusan National University, Busan, Republic of Korea, in 2017 and 2024, respectively. Since 2024, he has been a researcher with Cloud Security Research Center, Pusan National University. He is currently a visiting scholar with the Department of Computer Science and Engineering, University of California, Merced, USA. His research interests include privacy-preserving machine learning, vision transformer–based security, differential privacy, and AI-enhanced cryptography.



Jae-Seok Kim was born in Busan, South Korea in 1996. He received the BE and MS degrees, in 2021 and 2023, respectively, from Pusan National University, Busan, South Korea, where he is currently working toward the PhD degree in computer science and engineering. His research interests include anomaly detection and cloud security.



Seunghyuk Kim was born in Jeonju, South Korea in 1999. He received the BE degree in 2025 from Pusan National University, Busan, South Korea, where he is currently working toward the MS degree in computer science and engineering. His research interests include automated penetration testing and cloud security.



Jaeyoung Jeong was born in Daejeon, South Korea in 2001. She received the BE degree, in 2025 from Pusan National University, Busan, South Korea, where she is currently working toward the MS degree in computer science and engineering. Her research interests include cloud native and network security.



Yoon-Ho Choi (Member, IEEE) received the MS and PhD degrees from Seoul National University, Seoul, Korea, in 2004 and 2008, respectively. From 2008 to 2008, he was a postdoctoral scholar with Seoul National University. From Jan. 2009 to Dec. 2009, he was a postdoctoral scholar with Pennsylvania State University, University Park, PA, USA. He was a senior engineer with Samsung Electronics from 2010 to 2012. He was also deeply involved in the development of a commercial Long-Term Evolution Cloud Communication System. He was an assistant professor with Kyonggi University, Suwon, South Korea, from 2012 to 2014. He is currently an associate professor with the School of Computer Science and Engineering, Pusan National University, Busan, Korea. His research interests include cloud security, zerotrust, blockchain, anomaly detection algorithms, adversarial deep learning, and privacy preserving machine learning. Dr. Choi was a member of several technical program committees in various international conferences and journals.

Authorized licensed use limited to: VIT University. Downloaded on October 05,2026 at 15:35:22 UTC from IEEE Xplore. Restrictions apply.